

Agentic AI: The building blocks of sophisticated AI business applications

Received (in revised form): 1st May, 2024



Chhavi Chawla

Delivery Manager, MathCo, India

Chhavi Chawla is a data science professional with around nine years' experience in the analytics space. In MathCo, she leads delivery projects in generative AI across different domains of fast-moving consumer goods (FMCG), automotive and health.

MathCo Pvt. Ltd, 7th Floor West Wing, No. 13, Raheja Towers, Yellappa Chetty Layout, Bengaluru, Karnataka, India – 56001

E-mail: chhavi.chawla@mathco.com



Siddharth Chatterjee

Associate Principal, MathCo, India

Siddharth Chatterjee is a data science and machine learning (ML) graduate from UCL with more than nine years of experience in the analytics space. He leads the innovation team at MathCo in developing cutting-edge artificial intelligence products for their clients.

MathCo Pvt. Ltd, 7th Floor West Wing, No. 13, Raheja Towers, Yellappa Chetty Layout, Bengaluru, Karnataka, India – 56001

E-mail: siddharth.chatterjee@mathco.com



Sanketh Siddanna Gadadinni

Senior Associate, MathCo, India

Sanketh Siddanna Gadadinni holds a Bachelor's degree in electronics and communication. With four years of experience, he specialises in building natural language processing (NLP) products and generative AI (GenAI) applications.

MathCo Pvt. Ltd, 7th Floor West Wing, No. 13, Raheja Towers, Yellappa Chetty Layout, Bengaluru, Karnataka, India – 56001

E-mail: sanketh.gadadinni@mathco.com



Pulkit Verma

Manager, MathCo, India

Pulkit Verma has over six years of experience in innovating products through artificial intelligence. He completed his Master's and Bachelor's degrees in the field of computer science engineering.

MathCo Pvt. Ltd, 7th Floor West Wing, No. 13, Raheja Towers, Yellappa Chetty Layout, Bengaluru, Karnataka, India – 56001

E-mail: pulkit.verma@mathco.com



Sourav Banerjee

Partner, MathCo, India

Sourav Banerjee is a Partner at MathCo, a company at the forefront of the generative AI landscape. With a focus on creating artificial intelligence (AI)-driven products tailored to customer needs, he is dedicated to developing a unique suite of offerings in GenAI. His expertise extends across several key areas of AI, including natural language processing, computer vision and forecasting.

MathCo Pvt. Ltd, 7th Floor West Wing, No. 13, Raheja Towers, Yellappa Chetty Layout, Bengaluru, Karnataka, India – 56001

E-mail: sourav.banerjee@mathco.com

Abstract Language models (LMs) like GPT-4's ability to predict word sequences have made tasks such as summarisation and translation significantly easier. They often struggle, however, with complex reasoning tasks that require deliberate, multi-step processes. To address these limitations, the concept of agentic artificial intelligence (agentic AI) is introduced, where LMs are organised into workflows that mimic human-like iterative reasoning. This paper explores the four pillars of agentic AI frameworks: tool use, reflection, planning and multi-agent collaboration (MAC). Tool use allows LMs to access external resources such as search engines to enhance accuracy; reflection enables self-correction and iterative feedback; planning ensures the structured breakdown of tasks; and MAC involves multiple LMs collaborating on specific subtasks. The paper then compares and contrasts various MAC frameworks based on the inclusion of tool use, reflection and planning abilities, as well as the flow and control of information between different agents. Finally, potential use cases for MAC frameworks are discussed.

KEYWORDS: agentic AI, language models, multi-agent collaboration, complex problem solving

DOI 10.69554/XEHZ1946

INTRODUCTION

Language models (LMs) are essentially machine learning (ML) models designed to predict the next word in a sentence.¹ They are important for a variety of tasks, such as text generation,² text translation³ and answering questions.⁴ The expectation from LMs is that the language they generate is both extensive, covering all topics of human language, and human-like in persona. Achieving this requires LMs to be trained on large corpora of data with substantial computational power, resulting in models with millions or even billions of parameters. These models are known as large language models (LLMs). Examples include GPT-3 and GPT-4 from OpenAI, BLOOM from

BigScience, LLaMA from Meta and T5 (Text-to-Text Transfer Transformer) from Google.⁵

To understand how present-day LLMs generate text, consider an example of a text summarisation task that involves creating a concise understanding of a long text. The summaries generated in such tasks are of two kinds: (1) extractive in nature, ie summarising important excerpts and distilled versions of the main article; and (2) abstractive in nature, ie generating new text from an existing text corpus.⁶ The former kind are accurate but may suffer from issues of incoherence and redundancy, which is characteristic of deep learning (DL) models.⁷ On the other hand, the latter kind is how

present-day LLMs generate summaries. The abstractive and generative capabilities of LLMs give it relevance and human-like persona in natural language tasks such as text summarisation and can be leveraged via tools such as OpenAI's ChatGPT web interface.

Typically, an LLM task is carried out by providing an instruction such as 'summarise the following text [input]', a process known as prompting. Prompting involves crafting instructions in a way that elicits the desired output from the LLM.⁸ Sometimes the LLM may fail to interpret the instruction correctly, however, requiring a template or an example to clarify how the task should be executed. (Prompting without examples is called zero-shot prompting, while using two or more examples is referred to as few-shot prompting.⁹)

Tasks achieved using prompting techniques with LLMs are analogous to System 1 level of thinking¹⁰ displayed by the human mind when performing routine tasks that do not require much conscious thought.¹¹ These tasks are fast but prone to errors. The level of complexity provided by prompting, however, is not sufficient for more advanced tasks that involve reasoning

and logical deductions, which are more aligned with System 2 level of thinking (see Figure 1).

Most natural language tasks require a deeper level of processing akin to System 2 thinking.¹² To reach this level of complexity, a workflow is needed that approaches problems in the same way the human mind does.

AGENTIC WORKFLOW

For a better understanding, consider the example of task summarisation. As illustrated in Figure 2, this process does not happen all at once; instead, it follows a step-by-step approach whereby an outline is first created, then researched using external tools, rethought and corrected, finally producing a polished draft.

This method of reasoning and using LLMs with System 2 level thinking is known as an agentic workflow.¹³ Tools with agentic-like properties capable of achieving complex goals are referred to as agents.

In an agentic setup, multiple LLMs are dedicated to different subtasks, each generally associated with one or more attributes, such

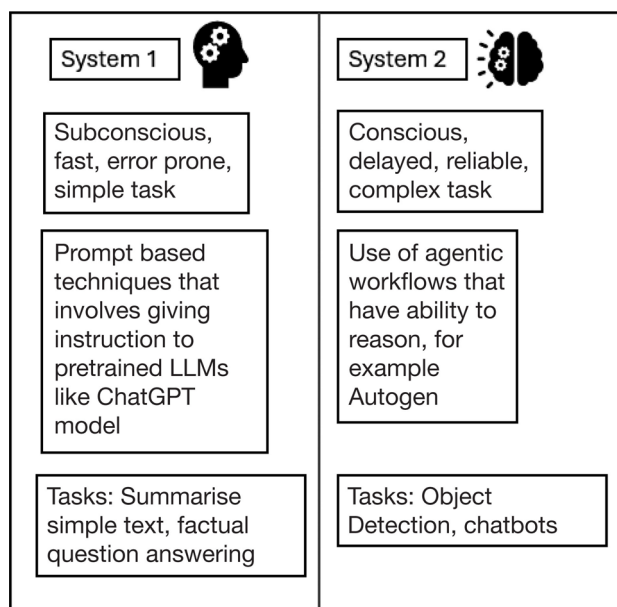


Figure 1: Difference between System 1 and System 2 level of tasks in LLMs



Figure 2: Iterative nature of text summarisation task

as planning the workflow, deciding when to communicate with external tools, self-error correction or self-feedback, and finally, managing and coordinating the subtasks to achieve the overall goal. The capability of an agent is defined by the presence or absence of these features. To make an informed decision about which agent to use for a System 2 level task, it is important to explore the four main dimensions that constitute an agent. These are:¹⁴

- *Tool use:* The use of external tools, such as access to web pages and external functions, to accomplish subtasks and, ultimately, the final output.
- *Reflection:* The ability of an LLM to reflect on mistakes and correct its own output. An example could be a code generation system with autonomous testing and correction capabilities.
- *Planning:* The ability of an LLM to independently decide the sequence of steps required to achieve a larger task, such as solving a mathematical word problem.
- *Multi-agent collaboration (MAC):* A multi-LLM environment where each LLM functions as an agent, and all these agents establish a messaging and communication system to solve a problem collaboratively.

There are many agentic applications, some of which incorporate all these features, while others include only a subset. The following sections will begin by discussing those that are rudimentary in design and will then move on to more complex workflows that encompass all the dimensions.

TOOL USE

Tool use is an ability of an LLM to enhance its querying capability by seeking help from external tools such as web applications to produce more accurate results.¹⁵ It was quickly recognised that relying solely on pre-trained or fine-tuned LLMs is limiting because they may not generate accurate answers for unseen text and can hallucinate (ie provide inaccurate answers that do not exist).¹⁶ Therefore, the need arose to equip LLMs with access to the latest information, which they might otherwise lack due to limitations in training them on the most recent corpus. The retrieval-augmented generation (RAG) technique helps address this problem by maintaining a knowledge base with task-specific information that can be retrieved and combined with prompts to produce context-specific results.¹⁷

While tool use with RAG is limited to an internal database, and even though the retrieved knowledge is relevant, it may still be outdated. In such cases, RAG can be further enhanced with function calling. The function call feature allows developers to translate prompts into structured queries.¹⁸ This structure helps map the task to the appropriate procedure (such as pre-written functions for a specific task, like text summarisation). This feature is particularly useful for connecting LLMs to external tools, applications, databases and code (see Figure 3).

There are challenges associated with function calling, however, particularly in accurately mapping the current function definition and supplying the

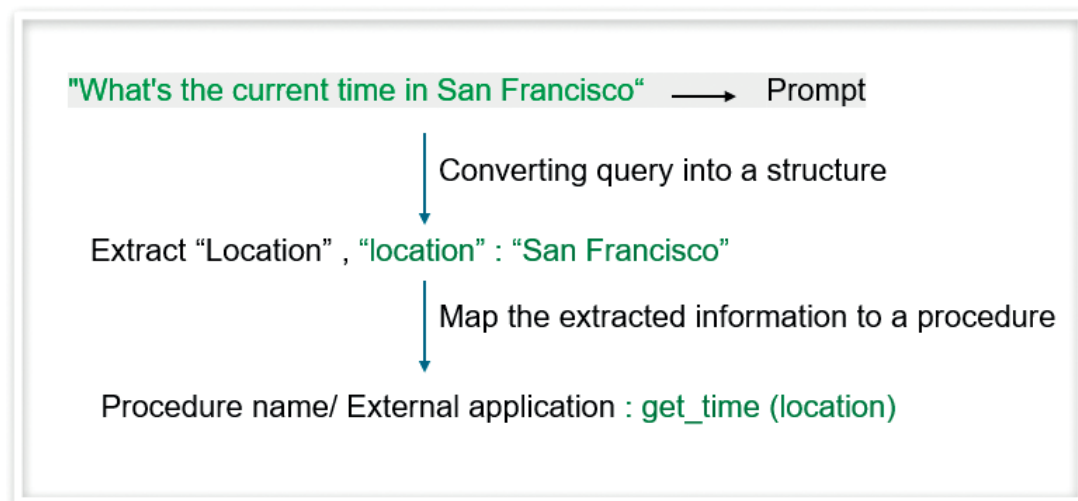


Figure 3: Conversion of natural language to a structured query which is then mapped to a predefined procedure or an external application

correct structured query to generate an appropriate response. Therefore, in addition to augmenting itself with up-to-date knowledge, agents should also exhibit intent recognition: the ability to identify the task being requested — a property known as self-reflection. The next section discusses reflection as an agentic property in detail.

REFLECTION

The ability of an agentic workflow to revisit and correct its decisions to produce the desired result is known as reflection.¹⁹ This self-feedback property empowers agents with the autonomy to change decisions and execute them accordingly. Reflection varies in complexity, ranging from basic to more sophisticated schemes. It can be roughly classified into three categories based on the number of agents employed and whether other agentic features, such as tool use, are integrated into the design.

Basic reflection involves two types of LLM calls: (1) generator — responsible for generating a response to the user prompt, and (2) critic — responsible for continuously correcting the response generated by the generator.²⁰ The feedback loop between the generator and the critic runs for a fixed

number of iterations until a satisfactory output is produced.

The key difference between basic reflection and reflection with tool use (see Figure 4), such as the reflexion model proposed by Shinn *et al.*,²¹ is that reflection can critique the responses generated by seeking help from external sources, ie employing tool use in addition to its self-reflection capability. Reflection generates responses and then gathers additional information and reasoning from the web. This positions reflection as a more advanced feature of agentic workflow compared to basic reflection. Increased complexity and reasoning ability in agents come with trade-offs in compute power and the number of LLM calls required in the feedback loop to achieve the desired accuracy. For tasks that are reasoning-intensive and require constant feedback, such as code generation and debugging, having reflection-like capabilities is essential in an agentic workflow.

Language agent tree search (LATS), developed by Zhou *et al.*,²² is an agentic workflow that employs not only reflection but also evaluation and searching mechanisms. LATS is a classic reinforcement learning (RL) inspired framework that

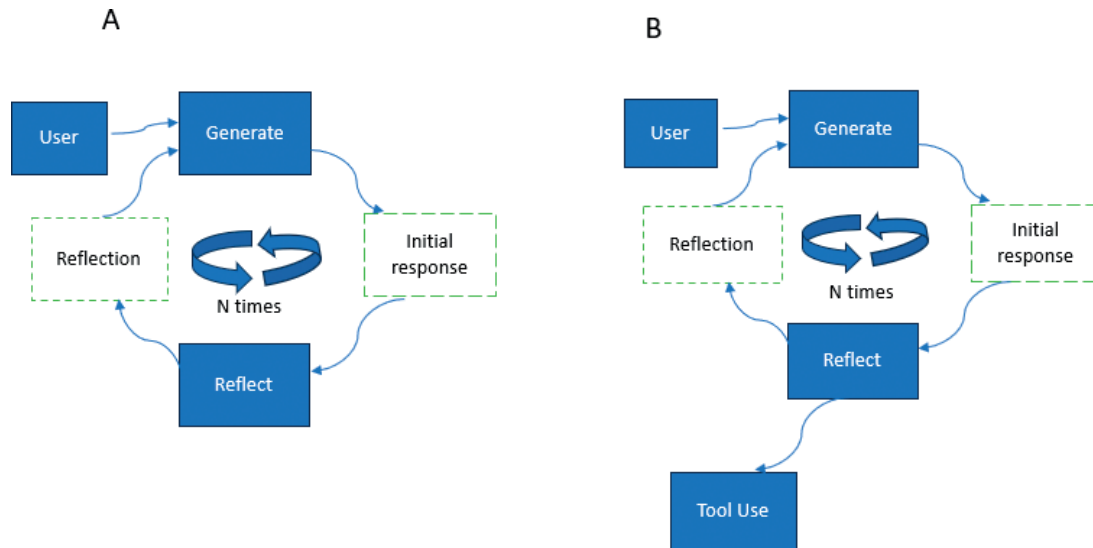


Figure 4: Depiction of basic (A) and (B) reflection with tool use schemes. The fundamental difference is the presence of tool use — as additional agentic feature — to the scheme

continuously evolves to find the best action given a particular state and the reward associated with it. The key difference from traditional RL is that the RL logic in LATS is replaced by an LLM network arranged in a tree-like structure, with the roots continuously updated to reflect the best action that maximises the reward. Figure 5 depicts a typical LATS design.

Delving further into the complexities of reflection, there are niche-based agents whose architectures are fine-tuned for specific domain tasks. Reflection-based agents designed specifically for translation and recommendation tasks are as follows:

- *Translation:* Reflective agents such as DUAL learning enhanced auto-REFLECTive Translation (DUAL REFLECT)²³ have evolved reflective capabilities specifically for handling translation tasks. These agents use a network of LLMs, each dedicated to draft translation, back translation, assessment and comparison. The reflective ability arises from comparing the back translation with the original text to identify biases. This bias, used as feedback, enhances the accuracy of translation tasks.

- *Session-based recommendation:* Session-based recommender systems (SRs) are designed to prioritise the present session when generating the next recommendation. SRs require an understanding of dynamic user interactions rather than static ones. Re2LLM, as a reflective agent, enhances its recommendation capabilities by first helping to build a correct knowledge base with hints and then selecting the most relevant hints based on task-specific feedback, guiding LLMs to improve their reasoning and recommendation abilities.²⁴

Reflection is one of the most critical capabilities of an autonomous agent; however, even reflection can fail in certain situations, such as when a reflective agent struggles to decipher complex tasks, particularly those involving machine interactions for writing code and instructions. Reflective agents can encounter the problem of being caught in the loop. Complex workflows require agents to decompose tasks into subtasks and then generate a plan, incorporating self-reflection, to address them. The following section discusses the need for planning as a feature in autonomous LLM agents.

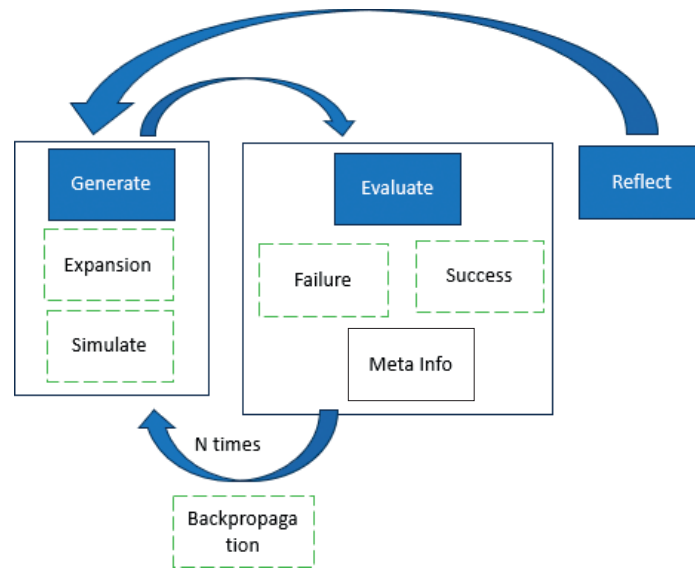


Figure 5: The first step is to generate action trees for the task through expansion and simulate processes. The next step is to evaluate the trees generated via simulation for their success or failure based on whether they reached the terminal goal. Information on success and failure is logged as meta info. The final step uses the meta info to reflect on it and suggest new actions for the generate module.

PLANNING

Tasks given to LLMs, such as ChatGPT, are often fixed and deterministic, where dynamic decision making and human intervention are unnecessary. A fixed set of commands can be provided, with the expectation that the task will be completed. For instance, asking an LLM to perform sentiment classification on restaurant reviews as either positive or negative is a straightforward task. More complex tasks, however, such as question answering or acting as a sales/travel agent, require dependent conversations that necessitate planning and execution flow. Planning, as a third domain of agentic workflow, introduces the ability to design an action plan and autonomously debug and adjust the course of action based on the system's state.²⁵ This process is intricate and involves utilising other agentic features.

For example, the task of downloading papers from an archive and filtering those important to the user might initially seem like a straightforward, deterministic process. Nevertheless, complexity arises at various stages: (1) autonomously creating a plan for executing the task; (2) breaking it down

into subtasks; (3) coding the instructions in a scripting language that the machine can understand; (4) choosing the appropriate action given the current state; and (5) reflecting on the process (see Figure 6).

System 1 (non-reasoning) plans can be generated through prompting, as discussed earlier. As task complexity increases, however, standard prompting may fail to break down the task effectively, leading to inaccurate results. In such cases, it is beneficial to design prompts that explicitly instruct the LLM to break complex tasks into subtasks. The simplest method for task decomposition is the chain of thought (COT) approach,²⁶ which uses a natural language prompt template to break down the task (see Figure 7). While some tasks require decomposition, others involve multiple steps toward a final goal, necessitating summarisation rather than decomposition.

Even though agents can decompose or summarise high-level plans as needed, a gap remains when translating natural language instructions into machine-level code. Agents such as ProAgent address this by using scripted languages such as JSON and Python

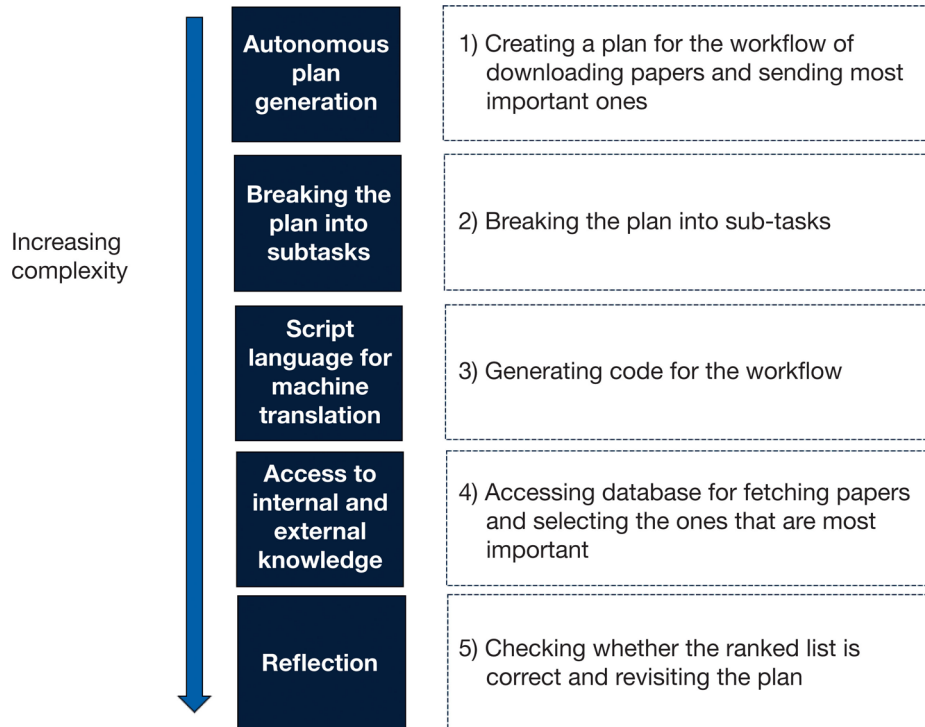


Figure 6: Complexity in planning tasks (top to bottom), showing autonomous plan generation as the least complex and reflection as the most complex task

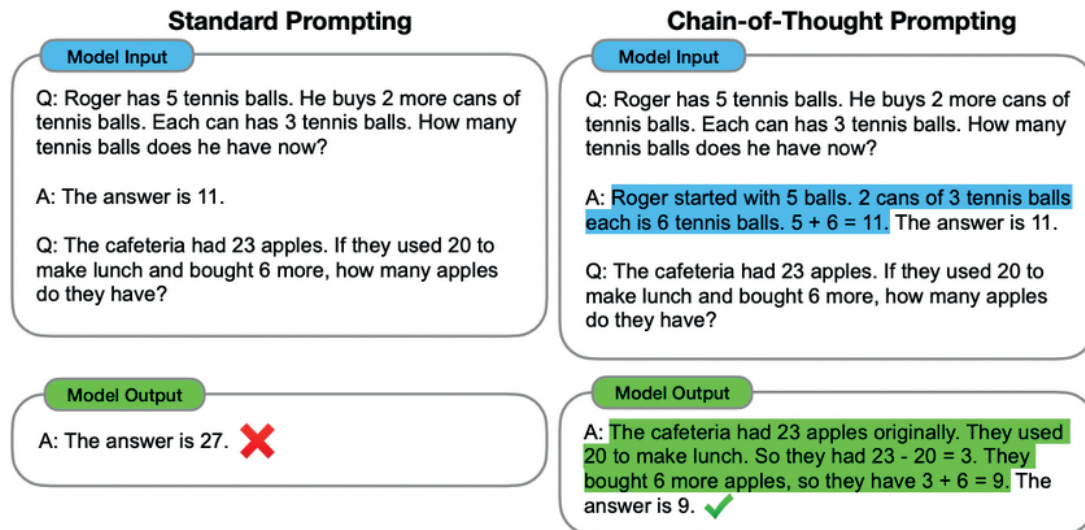


Figure 7: Standard prompting versus CoT prompting. The difference is highlighted by the standard prompting being unable to arrive at the answer to a mathematical problem while CoT arrives at the answer by breaking the prompt into individual statements

to write instruction templates and manage the overall workflow.²⁷

The next level of planning agents can do all the above while also leveraging an internal

knowledge base during plan formulation, ensuring that planning instructions are accurate and that the plan's integrity is maintained, avoiding hallucinations. The

KnowAgent framework improves upon ProAgent by incorporating internal tool use.²⁸

When planning, it is also important for agents to be aware of their environment to adjust the plan in response to changes in external conditions. In this context, many RL-based planners are useful for ensuring the correct mapping of actions to tasks, utilising historical observations.²⁹

The most advanced planning agents are those that also incorporate reflection, along with all the other attributes discussed. Examples include LLM systems that maintain a memory of past interactions to guide agents in providing self-feedback.

Planning can be complex and lengthy, depending on the task. While some LLMs can accept long input contexts, their ability to understand such input is mixed. They often face a ‘lost in the middle’ problem,³⁰ where the beginning and end context is retained, but the middle context is lost. To address this, individual agents can be assigned to each planned step and organised in a MAC framework.

MULTI-AGENT COLLABORATION

Consider a complex problem like writing working software code. The planner must define requirements, design the architecture, write the code, test it and then deploy it. This can be challenging for a single LLM call. A solution is to deploy a MAC framework that assigns each subtask to individual agents with specific roles. These roles can include real-life positions such as software engineer, product manager or quality tester, as well as abstract roles such as code interpreter or group chat manager.³¹

The interaction of multiple agents can lead to a complex array of LLM calls and messages, resulting in intricate workflows. Various emerging MAC frameworks manage these interactions in unique ways to address complex problems. In the context of software development, one such framework particularly suited for this task is MetaGPT (see Figure 8).

MetaGPT assigns roles such as product manager, architect, engineer and quality assurance (QA) engineer, each handling specific tasks. The product manager creates a product requirements document (PRD), which the architect translates into system designs. Engineers implement these designs after receiving them as tasks from the project manager and QA engineers test the code. This sequential, role-specific approach ensures that tasks are completed efficiently and accurately, minimising errors and enhancing the quality of the final software product.³²

MetaGPT employs hand-crafted roles and descriptions for various LLM-driven software agents to establish MAC. This approach involves a straightforward hand-off of work from one agent to the next, with no conversation occurring between agents. Each agent works individually on its task and then passes it to the next role in the sequence. The establishment of roles in the workflow is determined by the concept of SOP. The approach helps define the roles relevant for the software development task as exemplified in Figure 8.

An important aspect of real-world software development is its iterative nature, where different roles continuously interact through progress reviews and feedback.³³ MetaGPT’s unidirectional conversation flow does not support this back-and-forth

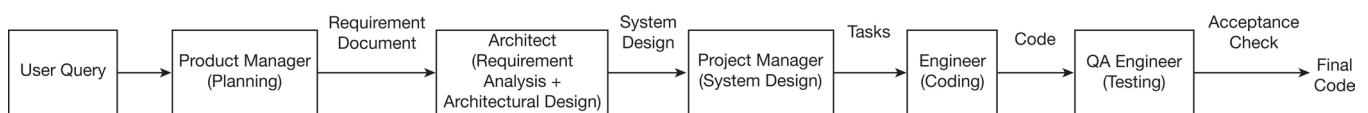


Figure 8: MetaGPT’s workflow, where software development tasks are managed through a structured process that mimics real-world standard operating procedures (SOPs)

collaboration. A MAC framework that supports greater inter-agent conversation is ChatDev.

ChatDev has a similar linear and role-based approach to software development as MetaGPT but it allocates two agents for each subtask as opposed to one (see Figure 9).

The process starts with a user query, which triggers the planning phase handled by the chief executive officer (CEO) (instructor) and chief technology officer (CTO) (assistant). The task then progresses to coding, where the CTO becomes the instructor and a programmer assists. Following coding, the reviewer (instructor) and programmer review the code, and finally, the tester (instructor) and programmer conduct testing. Each phase involves iterative back-and-forth communication between instructor and assistant to refine outputs, leading to the creation of the final code.³⁴

At each step, the two agents take on the roles of either assistant or instructor. The instructor initiates the conversation by specifying the task requirements, essentially telling the assistant, 'Here is what we need to create'. The assistant responds by attempting to create the desired output. Similar to basic reflection, the instructor then critiques the output of the assistant and provides feedback. Their conversation continues sequentially, iteratively improving the result until the instructor determines a sufficient output has been achieved, which is then passed to

the next pair in the conversation chain,³⁵ as shown in Figure 9.

The ChatDev MAC system has delivered better results than MetaGPT in software development benchmarks in terms of quality and consistency;³⁶ however, there are certain drawbacks to this framework. It has been observed that one way to improve performance during multiple LLM calls and prevent hallucination is by allowing humans to review and intervene during intermediate steps.³⁷ Currently, ChatDev does not provide for this intervention. Additionally, software development in companies is not a sequential process; subtasks and roles can be further subdivided and parallelised. For example, one developer might work on building the data pipeline while another simultaneously works on the model pipeline. Neither ChatDev nor MetaGPT properly handle situations where there is no interdependency between subtasks.

TaskWeaver is a multi-agent framework that tries to solve both these drawbacks with its more flexible conversation management architecture (see Figure 10). Unlike MetaGPT and ChatDev, TaskWeaver does not come with standard software development roles. Instead, it features two types of agent: planners and code interpreters. The planner acts as the user's entry point, breaking down requests into subtasks. The code interpreter generates code snippets relevant to the subtasks and reflects

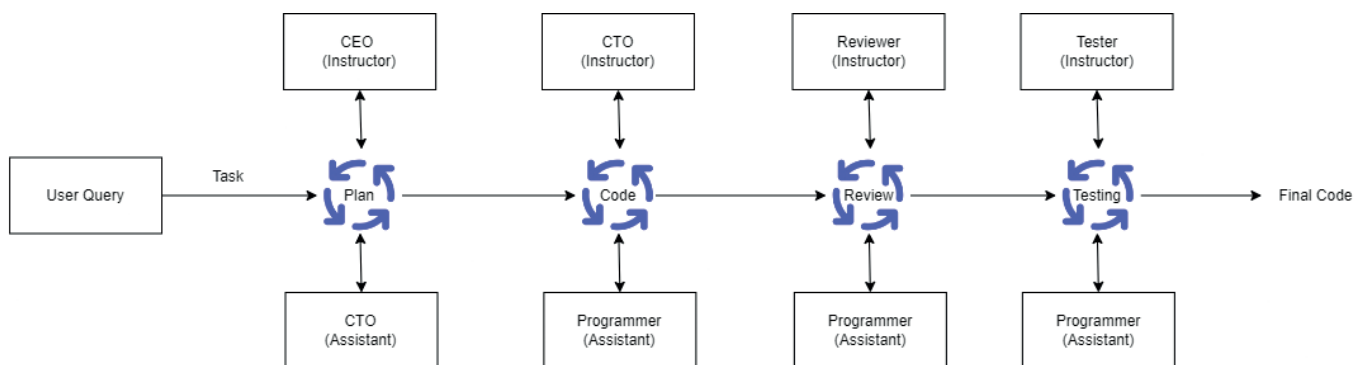


Figure 9: The ChatDev workflow, where specialised agents collaborate across different phases to develop software

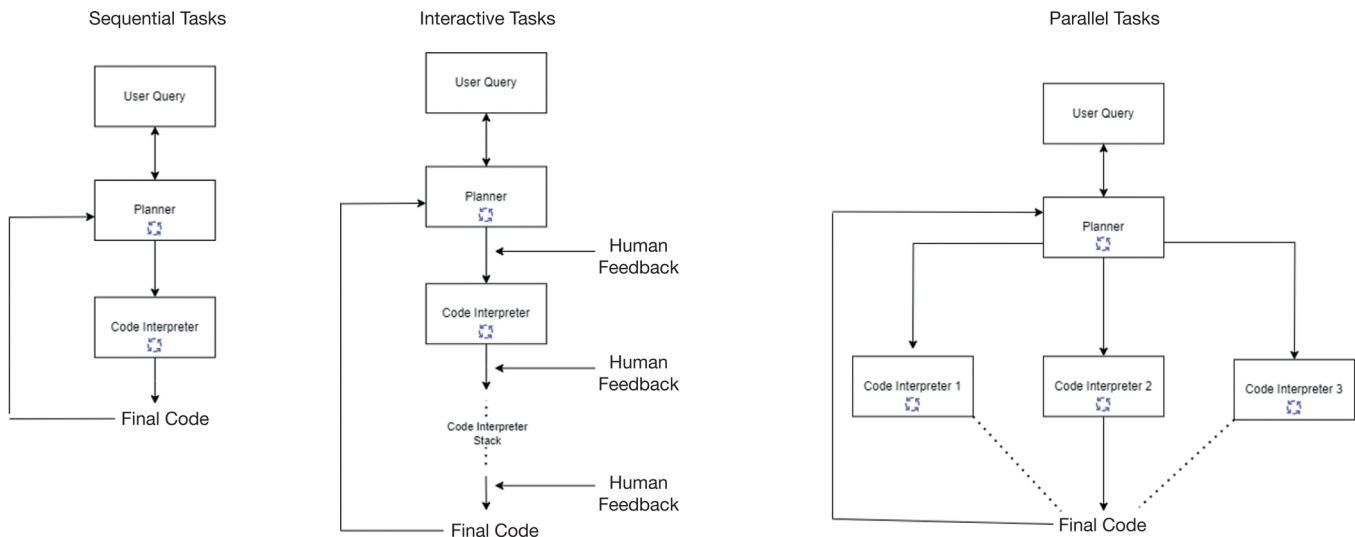


Figure 10: Dependencies between subtasks in TaskWeaver, managed by the planner and executed by the code interpreter

on the execution success. The final code is sent back to the planner to reflect if it met the subtasks' requirements. If satisfied, the planner presents the results to the user, or the ideation continues.

Figure 10 demonstrates how the planner breaks down the user query into subtasks and determines their dependencies. On the left, sequential tasks are shown, where each task depends on the completion of the previous one. The middle section depicts interactive tasks, which require human feedback between steps, involving a stack of interactions. On the right, parallel tasks are represented, where multiple tasks are executed simultaneously by different code interpreters, allowing for concurrent processing, ultimately combining into the final output. The code interpreter can self-reflect, while the planner reflects by comparing its own output with the final code before returning it to the user.³⁸

Depending on the task, the planner can decompose a user's request into three types of subtasks: sequential, interactive and parallel.³⁹ Sequential tasks, like those in MetaGPT and ChatDev, use the output of each task as input for the next. Interactive tasks are also sequential but allow human

feedback after each subtask, incorporating both the instruction and the previous output as context for the next step. Parallel subtasks, however, do not depend on each other for execution. Based on user requirements, the planner selects the most appropriate code interpreter for each subtask, allowing multiple subtasks to be executed in parallel (as shown in Figure 10).

A common theme across the MAC frameworks reviewed so far is that they restrict conversations to within the agent or a limited number of other agents. None of these frameworks offer a scalable approach for managing conversations among several agents simultaneously.

Autogen is a powerful MAC framework with a conversation management system capable of scaling across numerous agents.⁴⁰ It achieves this through three building blocks that can be flexibly arranged: (1) an assistant agent performing various tasks such as reflection, planning or tool use; (2) a human proxy agent providing feedback after subtasks are executed by the assistant agent, which can be an actual human or an LLM simulating human review; and (3) a group chat manager determining which agent speaks and which listens.

The assistant agent and human proxy agent can be configured to behave similarly to the agents seen in the earlier frameworks. The group chat manager, however, is a unique addition to this framework, acting as a conductor for conversations among multiple agents. It operates through a three-step process: first, the group chat manager selects the agent that needs to speak; next, it determines which other agents need to hear the message; finally, the message is broadcasted to those selected agent. This orchestrated approach ensures effective communication and coordination across all agents.⁴¹

Figure 11 provides an example of the orchestrated approach. Assistant Agent 3 is prompted to speak by the manager while Agents 1 and 2 receive the input. This process can repeat multiple times until the issue is resolved. The assistant agent can also be replaced by human proxy agents if necessary.⁴²

While GroupChatManager in Autogen is effective at selecting the next speaker based on context and roles, one of its limitations is that it lacks a built-in mechanism to stop or interrupt the chat, making it difficult to

manage conversation flow when immediate control is needed.

DISCUSSION

Agentic AI and MAC frameworks represent the future of generative AI (GenAI) by enabling models to operate iteratively, leverage external tools and work together on complex tasks. This approach enhances performance and efficiency, surpassing the capabilities of even the most advanced standalone models like GPT-4,⁴³ paving the way for more intelligent and adaptable artificial intelligence (AI) systems.

Given the distinct capabilities of various multi-agent frameworks, businesses should carefully evaluate their operational needs before selecting the most suitable framework for their applications. For instance, MetaGPT is effective for tasks with predefined workflows and could optimise a multinational corporation's supply chain. In this scenario, each node of MetaGPT represents different agents, such as suppliers, warehouses, transportation and retailers. These agents follow standard operating procedures to predict inventory needs,

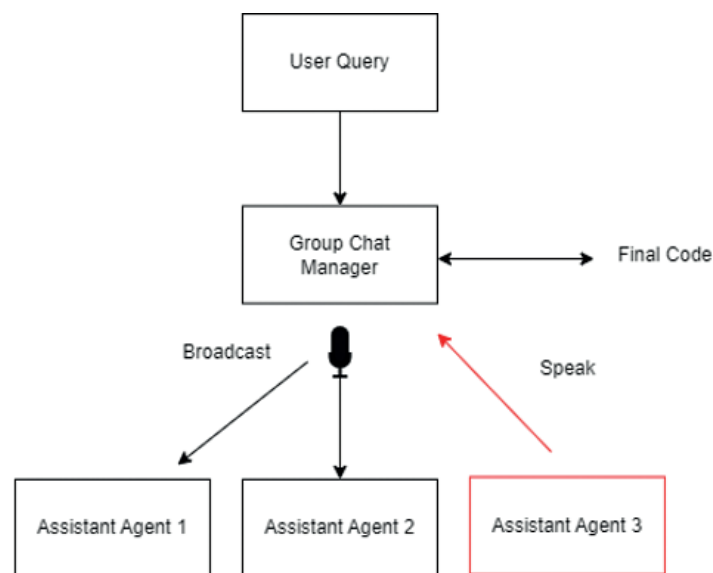


Figure 11: Dynamic group chat demonstrates how to build a versatile group chat using Autogen

manage restocking and coordinate logistics as data becomes available.

Similarly, ChatDev, with its instructor–assistant framework and the ability to reflect at each step, is well suited for companies needing strict regulatory compliance. For example, a healthcare provider developing a comprehensive patient management system that integrates appointment scheduling, medical records and billing must avoid Health Insurance Portability and Accountability Act (HIPAA) violations. Instructor nodes can review the work of assistant nodes at every step to ensure compliance.

TaskWeaver, which incorporates real-time human feedback, can serve as an effective personalised financial advisory tool for bank clients. A client might input their financial status and goals, prompting the advisory engine to suggest investment portfolios. The client can then provide additional details about their risk tolerance, allowing TaskWeaver to refine and offer a portfolio tailored to the client's preferences and objectives. This dynamic interaction enables the engine to adapt recommendations in response to evolving client inputs, ensuring more precise and actionable financial advice.

Autogen's GroupChatManager, with its conversation scaling capabilities, can enhance customer service by transforming how service departments handle customer interactions. In this setup, the GroupChatManager coordinates a team of specialised agents trained in areas such as billing, technical support and product enquiries. For example, if a customer presents a technical issue, the technical support agent initially addresses it. If the issue also involves a billing concern, the GroupChatManager introduces the billing agent into the conversation. This integrated approach could ensure that customer issues are resolved efficiently and comprehensively, without requiring multiple transfers or callbacks.

In summary, the building blocks of agentic AI, combined with iterative workflows and multi-agent collaboration, significantly enhance AI capabilities, bridging the gap between simple task execution and complex problem solving. These frameworks can be tailored to various operational needs, enabling more adaptive and precise AI-driven solutions. As AI continues to evolve, the strategic selection and implementation of these frameworks will be critical to realising their full potential across diverse industries, ensuring that AI systems not only perform tasks efficiently but also adapt to the complexities of real-world applications.

References

1. Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I., 'Improving Language Understanding by Generative Pre-Training', available at https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf (accessed 12th August, 2024).
2. Arcas, Y. and Agüera, B. (May 2022), 'Do Large Language Models Understand Us?', *Daedalus*, Vol. 151, No. 2, pp. 183–197.
3. Moslem, Y., Haque, R., Kelleher, J. D. and Way, A. (May 2023), 'Adaptive Machine Translation with Large Language Models', arXiv, available at <http://arxiv.org/abs/2301.13294> (accessed 12th August, 2024).
4. Adiwardana, D., Luong, M.-T., So, D. R., Hall, J., Fiedel, N., Thoppilan, R., Yang, Z., Kulshreshtha, A., Nemade, G., Lu, Y. and Le, Q. V. (February 2020), 'Towards a Human-like Open-Domain Chatbot', arXiv, available at <http://arxiv.org/abs/2001.09977> (accessed 12th August, 2024).
5. Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y. and Wen, J.-R. (November 2023), 'A Survey of Large Language Models', arXiv, available at <http://arxiv.org/abs/2303.18223> (accessed 12th August, 2024).
6. Giarelis, N., Mastrokostas, C. and Karacapilidis, N. (June 2023), 'Abstractive vs. Extractive Summarization: An Experimental Review', *Applied Sciences*, Vol. 13, No. 13, 7620.
7. Verma, S. and Nidhi, V. (2019), 'Extractive Summarization using Deep Learning', arXiv, available at <http://arxiv.org/abs/1708.04439> (accessed 12th August, 2024).
8. Chang, K., Xu, S., Wang, C., Luo, Y., Xiao, T. and Zhu, J. (April 2024), 'Efficient Prompting Methods for Large Language Models: A Survey', arXiv,

- available at <http://arxiv.org/abs/2404.01077> (accessed 12th August, 2024).
9. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Snyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I. and Amodei, D. (July 2020), 'Language Models Are Few-Shot Learners', arXiv, available at <http://arxiv.org/abs/2005.14165> (accessed 12th August, 2024).
10. Kahneman, D. (2011), *Thinking, Fast and Slow*, Farrar, Straus and Giroux, New York.
11. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. and Iwasawa, Y. (2023), 'Large Language Models are Zero-Shot Reasoners', arXiv, available at <http://arxiv.org/abs/2205.11916> (accessed 12th August, 2024).
12. Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S. and Kambhampati, S. (November 2023), 'PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change', arXiv, available at <http://arxiv.org/abs/2206.10498> (accessed 12th August, 2024).
13. The Batch (2024), 'Four AI Agent Strategies That Improve GPT-4 and GPT-3.5 Performance', Deep Learning, available at <https://www.deeplearning.ai/the-batch/how-agents-can-improve-llm-performance/?ref=dl-staging-website.ghost.io> (accessed 12th August, 2024).
14. *Ibid.*
15. The Batch (2024), 'Agentic Design Patterns Part 3: Tool Use', Deep Learning, available at <https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-3-tool-use/?ref=dl-staging-website.ghost.io> (accessed 12th August, 2024).
16. Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., Wang, L., Luu, A. T., Bi, W., Shi, F. and Shi, S. (September 2023), 'Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models', arXiv, available at <http://arxiv.org/abs/2309.01219> (accessed 12th August, 2024).
17. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M. and Wang, H. (March 2024), 'Retrieval-Augmented Generation for Large Language Models: A Survey', arXiv, available at <http://arxiv.org/abs/2312.10997> (accessed 12th August, 2024).
18. OpenAI Platform, 'Open AI developer platform', available at <https://platform.openai.com> (accessed 12th August, 2024).
19. The Batch (2024), 'Agentic Design Patterns Part 2: Reflection', Deep Learning, available at <https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-2-reflection/?ref=dl-staging-website.ghost.io> (accessed 12th August, 2024).
20. LangChain (2024), 'Reflection Agents', blog, available at <https://blog.langchain.dev/reflection-agents/> (accessed 12th August, 2024).
21. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K. and Yao, S. (2023), 'Reflexion: Language Agents with Verbal Reinforcement Learning', arXiv, available at <http://arxiv.org/abs/2303.11366> (accessed 12th August, 2024).
22. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H. and Wang, Y. X. (2024), 'Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models', arXiv, available at <http://arxiv.org/abs/2310.04406> (accessed 12th August, 2024).
23. Chen, A., Lou, L., Chen, K., Bai, X., Xiang, Y., Yang, M., Zhao, T. and Zhang, M. (June 2024), 'DUAL-REFLECT: Enhancing Large Language Models for Reflective Translation through Dual Learning Feedback Mechanisms', arXiv, available at <http://arxiv.org/abs/2406.07232> (accessed 12th August, 2024).
24. Wang, Z., Du, Y., Sun, Z., Chua, H., Feng, K., Wang, W. and Zhang, J. (April 2024), 'Re2LLM: Reflective Reinforcement Large Language Model for Session-based Recommendation', arXiv, available at <http://arxiv.org/abs/2403.16427> (accessed 12th August, 2024).
25. The Batch (2024), 'Agentic Design Patterns Part 4: Planning', Deep Learning, available at <https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-4-planning/?ref=dl-staging-website.ghost.io> (accessed 12th August, 2024).
26. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (January 2023), 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', arXiv, available at <http://arxiv.org/abs/2201.11903> (accessed 12th August, 2024).
27. Ye, Y., Cong, X., Tian, S., Cao, J., Wang, H., Qin, Y., Lu, Y., Yu, H., Wang, H., Lin, Y., Liu, Z. and Sun, M. (November 2023), 'ProAgent: From Robotic Process Automation to Agentic Process Automation', arXiv, available at <http://arxiv.org/abs/2311.10751> (accessed 12th August, 2024).
28. Zhu, Y., Qiao, S., Ou, Y., Deng, S., Zhang, N., Lyu, S., Shen, Y., Liang, L., Gu, J. and Chen, H. (March 2024), 'KnowAgent: Knowledge-Augmented Planning for LLM-Based Agents', arXiv, available at <http://arxiv.org/abs/2403.03101> (accessed 12th August, 2024).
29. Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R. and Chen, E. (February 2024), 'Understanding the planning of LLM agents: A survey', arXiv, available at <http://arxiv.org/abs/2402.02716> (accessed 12th August, 2024).
30. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. and Liang, P. (January 2024), 'Lost in the Middle: How Language Models Use Long Contexts', *Transactions of the Association for Computational Linguistics*, Vol. 12, pp. 157–173.
31. The Batch, ref. 13 above.
32. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Ka Shing Yau, S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C. and Schmidhuber, J. (November 2023), 'MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework', arXiv, available at <http://arxiv.org/abs/2308.00352> (accessed 12th August, 2024).

33. Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z. and Sun, M. (June 2024), 'ChatDev: Communicative Agents for Software Development', arXiv, available at <http://arxiv.org/abs/2307.07924> (accessed 12th August, 2024).
34. Liu *et al.*, ref. 30.
35. *Ibid.*
36. *Ibid.*
37. Qiao, B., Li, L., Zhang, X., He, S., Kang, Y., Zhang, C., Yang, F., Dong, H., Zhang, J., Wang, L., Ma, M., Zhao, P., Qin, S., Qin, X., Du, C., Xu, Y., Lin, Q., Rajmohan, S. and Zhang, D. (June 2024), 'TaskWeaver: A Code-First Agent Framework', arXiv, available at <http://arxiv.org/abs/2311.17541> (accessed 12th August, 2024).
38. Liu *et al.*, ref. 30
39. *Ibid.*
40. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D. and Wang, C. (October 2023), 'AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation', arXiv, available at <http://arxiv.org/abs/2308.08155> (accessed 12th August, 2024).
41. Qiao *et al.*, ref. 37.
42. *Ibid.*
43. Smith, T. (June 2024), 'How AI Agentic Workflows Could Drive More AI Progress Than Even the Next Generation of Foundation Models', DZone, available at <https://dzone.com/articles/how-ai-agentic-workflows-could-drive-more-ai> (accessed 12th August, 2024).